

“Living Schematic” — a CAD-engine equivalent for schematics & diagrams (proposal)

Turn a flat scanned schematic into an interactive diagram with animated flow IN THE DIRECTION of the current/fluid, and click-through component breakdowns.

THE ANALOGY

CAD engine: a PART -> classify shape -> build procedural geometry -> rendered image (tiered, cached). Living Schematic: a SCHEMATIC PAGE -> extract its vectors+labels -> infer the connectivity graph (netlist) -> ANIMATE interactive diagram (tiered, cached). Same idea: reconstruct structured meaning from the source, then render something richer than the flat scan.

HOW IT WORKS — the pipeline

1 VECTORIZE

already have it:
schem_overlay.py / /api/schempaths
-> lines, curves + text boxes
(R1/R2/C1, net names, pin #s)

2 INFER GRAPH

NEW: snap line endpoints at
junctions -> nodes/nets; attach
labelled symbols to nodes
-> a netlist (graph) sidecar

3 SIMULATE

already have it:
Circuit Lab MNA solver
(circuitsim.js) -> node V,
branch currents + direction

4 ANIMATE

NEW: schemflow.js overlay —
dashes/particles travel each
wire in the CURRENT'S direction,
speed $\propto$ |current|

5 BREAK DOWN

NEW: click a component ->
its CAD image + part record
(NSN/part#), connected nets,
'trace this signal' walkthrough

green = building blocks that ALREADY EXIST · amber = the new pieces. Per-page graph cached like figcrop/cadcache (schemgraph.db sidecar, R1).

THE EXPERIENCE (what the mechanic sees)

Animated flow

Current/signal — or hydraulic/pneumatic fluid — flows along the wires/lines in its real direction. Play / pause / speed. Click a wire to zoom in.

Trace-a-net

Click a wire or a net name: the whole net lights up and the animation isolates to just that path, end to end, across the page.

Component breakdown

Click a symbol (R12, valve, relay): a panel shows its CAD image, NSN/part #, value/rating, the nets it touches, and why it's there.

Guided walkthrough

Step the signal source -> load in order (like the onboarding tour, on the schematic) — the 'directions of the directions' as a movie.

SCALES WITH THE BUILD (RPS), like CAD

MODERN

WebGL particle flow along traces + live re-simulation; smooth, many particles.

LITE

Lightweight animated SVG dashes (stroke-dashoffset); the same direction + speed.

LEGACY

No continuous animation: static highlight + a STEP button that advances the flow one step (RPS-safe, no rAF loop).

PROS

- Reuses what's built: schempaths + Circuit Lab + CAD images + RPS + the part records.
- Turns a static scan into a teaching tool — flow + direction is how techs actually reason.
- Generalises beyond circuits: hydraulic/pneumatic/fuel/coolant diagrams animate the same way (fluid flow).
- Ties every symbol to its real part (NSN) and its CAD image — one unified object model.
- Offline, deterministic, cached per page; degrades gracefully on legacy.

CONS / RISKS

- Graph inference from a SCANNED schematic is the hard part — OCR noise, dense/overlapping lines, hand drawing.
- Needs a human-in-the-loop review/override for wrong junctions (an append-only fix queue, like the NIIN review).
- Simulation needs component VALUES; many TM schematics omit them — animate topology/direction even without a full solve.
- Animation cost on big pages; cap particle counts + tier it; cache the graph, not the frames.
- Not every 'schematic' is a graph (block diagrams, exploded views) — detect type, fall back to the highlighter.

Suggested rollout (each step shippable on its own)

1) schemgraph.py + BUILD-SCHEMGRAPH.bat: host batch infers the netlist per schematic page -> schemgraph.db sidecar (with a confidence score). 2) /api/schemgraph route serves it. 3) schemflow.js: the animated overlay (tiered). 4) Component breakdown panel reusing /api/part_record + /cadimg. 5) Review/override queue for bad junctions. 6) Wire it into Circuit Lab + the schematics viewer. Proof-of-concept first on ONE clean wiring page to prove the graph inference, then scale.